

LAPORAN PRAKTIKUM
STRUKTUR DATA
“laporan Praktikum Pekan 5”

Disusun Oleh:

Faiz Fikri Satria

2511533026

Dosen Pengampu : Dr. Wahyudi, S.T, M.T.
Asisten Praktikum : Sherly Sukmadira



DAPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur kami panjatkan ke hadirat Tuhan Yang Maha Esa atas rahmat dan karunia-Nya sehingga kami dapat menyelesaikan praktikum dan laporan ini dengan baik. Laporan praktikum ini disusun sebagai dokumentasi dari praktikum pekan ke-5 mata kuliah Struktur Data yang membahas implementasi **Singly Linked List (SLL)** menggunakan bahasa pemrograman Java. Praktikum ini membahas pembuatan node, operasi penambahan (insert), pencarian dan traversal, serta penghapusan (delete) pada linked list. Melalui praktikum ini, kami mempelajari bagaimana struktur data linier yang dinamis bekerja tanpa batas ukuran tetap seperti array.

Kami mengucapkan terima kasih yang sebesar-besarnya kepada Bapak Dr. Wahyudi, S.T., M.T. selaku dosen pengampu yang telah memberikan materi dan arahan yang jelas, serta Sherly Sukmadira selaku asisten praktikum yang selalu sabar membimbing selama sesi praktikum. Terima kasih juga kepada rekan-rekan mahasiswa yang telah membantu dan berdiskusi dalam menyelesaikan tugas praktikum. Praktikum ini sangat bermanfaat untuk memahami konsep pointer, referensi objek, dan manajemen memori secara manual dalam Java.

Kami menyadari bahwa laporan ini masih memiliki kekurangan baik dari segi penulisan maupun teknis. Oleh karena itu, kritik dan saran yang membangun sangat kami harapkan untuk penyempurnaan laporan di masa yang akan datang.

Padang, 06 Mei 2026

Penulis

DAFTAR ISI

KATA PENGANTAR	ii
DAFTAR ISI	iii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan	1
1.3 Manfaat	1
BAB II PEMBAHASAN	2
2.1 Class Node	2
2.1.1 Teori	2
2.1.2 Langkah-langkah	2
2.1.3 Contoh Kode	2
2.1.5 Analisis.....	2
2.2 Class Pencarian	3
2.2.1 Teori	3
2.2.2 Langkah-langkah	3
2.2.3 Contoh Kode	3
2.2.5 Analisis	3
2.3 Class Penambahan	3
2.2.1 Teori	3
2.2.2 Langkah-langkah	4
2.2.3 Contoh Kode	4

2.2.4 Hasil	4
2.2.5 Analisis	4
2.4 Class Penghapusan	4
2.2.1 Teori	4
2.2.2 Langkah-langkah	5
2.2.3 Contoh Kode	5
2.2.4 Hasil	5
2.2.5 Analisis	5
2.6 Tugas Pekan 2	6
2.5.1. Analisis	7
BAB III KESIMPULAN	8
DAFTAR PUSTAKA	9

BAB I

PENDAHULUAN

1.1 Latar Belakang

Singly Linked List (SLL) merupakan struktur data linier yang terdiri dari serangkaian node dimana setiap node berisi data dan pointer ke node berikutnya. Berbeda dengan array, SLL bersifat dinamis sehingga ukurannya dapat bertambah atau berkurang sesuai kebutuhan selama runtime. Praktikum pekan ini membahas implementasi SLL secara manual tanpa menggunakan kelas bawaan Java.

Praktikum ini meliputi pembuatan class Node, operasi penambahan data di depan, di belakang, dan di posisi tertentu, serta operasi pencarian dan penghapusan node.

1.2 Tujuan

1. Memahami konsep dan struktur dasar Singly Linked List.
2. Mengimplementasikan class Node untuk merepresentasikan elemen linked list.
3. Mengimplementasikan operasi penambahan data (insert at front, end, dan position).
4. Mengimplementasikan operasi pencarian (search) dan traversal pada SLL.
5. Mengimplementasikan operasi penghapusan data (delete head, last, dan by position).

1.3 Manfaat

1. Mahasiswa mampu memahami perbedaan linked list dengan array serta kelebihanannya dalam hal fleksibilitas memori.
2. Meningkatkan kemampuan pemrograman dengan konsep pointer dan referensi objek.
3. Memberikan dasar yang kuat untuk mempelajari struktur data yang lebih kompleks seperti Doubly Linked List dan Tree..

BAB II

PEMBAHASAN

2.1 Class Node

2.1.1 Teori Node adalah komponen dasar dari Linked List. Setiap node terdiri dari dua bagian: data dan pointer (next) yang menunjuk ke node berikutnya.

2.1.2 Langkah-langkah

1. Membuat class `NodeSLL_2511533026`.
2. Mendeklarasikan variabel `data_3026` dan `next_3026`.
3. Membuat konstruktor untuk inisialisasi data dan pointer.

2.1.3 Contoh Kode

```
1 package Pekan5_2511533026;
2
3 public class NodeSLL_2511533026 {
4     // node bagian data
5     int data_3026;
6     // pointer ke node berikutnya
7     NodeSLL_2511533026 next_3026;
8     public NodeSLL_2511533026(int data_3026)
9     {
10         this.data_3026 = data_3026;
11         this.next_3026 = null;
12     }
13 }
14
```

2.1.4 Analisis

Class Node berfungsi sebagai blueprint untuk setiap elemen linked list. Penggunaan referensi `next_3026` memungkinkan pembentukan rantai node secara dinamis.

2.2 Class Pencarian

2.2.1 Teori

Operasi traversal digunakan untuk menelusuri seluruh node, sedangkan search digunakan untuk mencari keberadaan suatu nilai dalam linked list.

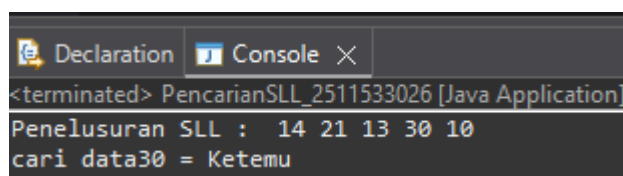
2.2.2 Langkah-langkah

1. Membuat method traversal untuk mencetak semua data.
2. Membuat method searchKey untuk mencari data tertentu.
3. Mengimplementasikan main untuk testing.

2.2.3 Contoh Kode

```
1 package Pekarib_2511533026;
2
3 public class PencarianSLL_2511533026 {
4     static boolean searchKey(NodeSLL_2511533026 head, int key) {
5         NodeSLL_2511533026 curr = head;
6         while (curr != null) {
7             if (curr.data_3026 == key)
8                 return true;
9             curr = curr.next_3026;
10        }
11        return false;
12    }
13    public static void traversal(NodeSLL_2511533026 head) {
14        //mulai dari head
15        NodeSLL_2511533026 curr = head;
16        //gunakan variabel pointer null
17        while (curr != null) {
18            System.out.print(" " + curr.data_3026);
19            curr = curr.next_3026;
20            System.out.println();
21        }
22    }
23    public static void main(String[] args) {
24        NodeSLL_2511533026 head = new NodeSLL_2511533026(14);
25        head.next_3026 = new NodeSLL_2511533026(21);
26        head.next_3026.next_3026 = new NodeSLL_2511533026(13);
27        head.next_3026.next_3026.next_3026 = new NodeSLL_2511533026(30);
28        head.next_3026.next_3026.next_3026.next_3026 = new NodeSLL_2511533026(10);
29        System.out.print("Penelusuran SLL : ");
30        traversal(head);
31        // data yang akan dicari
32        int key = 30;
33        System.out.print("cari data " + key + " = ");
34        if (searchKey(head, key))
35            System.out.println("ketemu");
36        else System.out.println("tidak ada");
37    }
38 }
```

2.2.4 Hasil Output



```
<terminated> PencarianSLL_2511533026 [Java Application]
Penelusuran SLL : 14 21 13 30 10
cari data30 = Ketemu
```

2.2.5 Analisis

Traversal memiliki kompleksitas $O(n)$, begitu juga dengan search. Implementasi ini efektif untuk linked list yang tidak terurut.

2.3 Class Penambahan

2.3.1 Teori Operasi insert pada SLL dapat dilakukan di depan (insertAtFront), di belakang (insertAtEnd), dan di posisi tertentu (insertPos).

2.3.2 Langkah-langkah

1. Implementasi insertAtFront.
2. Implementasi insertAtEnd dengan traversal untuk mencari node terakhir.
3. Implementasi insertPos untuk menyisipkan pada posisi tertentu.

2.3.3 Contoh Kode

```
1 package tatanan5_2511533026;
2
3 public class TatananSLL_2511533026 {
4     public static NodeSLL_2511533026 insertAtFront(NodeSLL_2511533026 head, int value) {
5         NodeSLL_2511533026 new_node = new NodeSLL_2511533026(value);
6         new_node.next_3026 = head_3026;
7         return new_node;
8     }
9
10    // fungsi penambahan node di akhir sll
11    public static NodeSLL_2511533026 insertAtEnd(NodeSLL_2511533026 head, int value) {
12        // buat node baru dengan atribut
13        NodeSLL_2511533026 new_node = new NodeSLL_2511533026(value);
14        // jika list kosong maka new_node akan
15        if (head == null) {
16            return new_node;
17        }
18        // akan used ke variabel tempNode
19        NodeSLL_2511533026 temp = head;
20        // selangkah ke node akhir
21        while (temp.next_3026 != null) {
22            temp = temp.next_3026;
23        }
24        // temp pointer
25        temp.next_3026 = new_node;
26        return head;
27    }
28
29    public static NodeSLL_2511533026 getHead(int data_3026) {
30        return new NodeSLL_2511533026(data_3026);
31    }
32
33    static NodeSLL_2511533026 insertPos(NodeSLL_2511533026 headNode, int position, int value) {
34        NodeSLL_2511533026 head = headNode;
35        if (position < 1) {
36            System.out.println("Invalid position");
37        }
38        if (position == 1) {
39            return insertAtFront(headNode, value);
40        }
41        // temp pointer
42        NodeSLL_2511533026 temp = head;
43        // selangkah ke node ke-1
44        while (position != 1) {
45            temp = temp.next_3026;
46            position--;
47        }
48        // temp pointer
49        temp.next_3026 = new NodeSLL_2511533026(value);
50        return head;
51    }
52
53    // fungsi untuk menampilkan list
54    public static void printList(NodeSLL_2511533026 head) {
55        System.out.println("Posisi di luar jangkauan");
56        return head;
57    }
58
59    public static void printList(NodeSLL_2511533026 head) {
60        NodeSLL_2511533026 curr = head;
61        while (curr.next_3026 != null) {
62            System.out.print(curr.data_3026 + "-->");
63            curr = curr.next_3026;
64        }
65        if (curr.next_3026 == null) {
66            System.out.print(curr.data_3026);
67            System.out.println();
68        }
69    }
70
71    public static void main(String[] args) {
72        // buat linked list 2-3-5-6
73        NodeSLL_2511533026 head = new NodeSLL_2511533026(2);
74        head.next_3026 = new NodeSLL_2511533026(3);
75        head.next_3026.next_3026 = new NodeSLL_2511533026(5);
76        head.next_3026.next_3026.next_3026 = new NodeSLL_2511533026(6);
77        // cetak list awal
78        System.out.println("Senarai berantai awal:");
79        printList(head);
80        // tambahkan node baru di depan
81        System.out.println("tambah 1 simpul di depan:");
82        int data = 1;
83        head = insertAtFront(head, data);
84        // cetak node list
85        printList(head);
86        // tambahkan node baru di belakang
87        System.out.println("tambah 1 simpul di belakang:");
88        int data2 = 7;
89        head = insertAtEnd(head, data2);
90        // cetak node list
91        printList(head);
92        // tambahkan node baru di posisi tertentu
93        System.out.println("tambah 1 simpul ke data 4:");
94        int data3 = 4;
95        head = insertPos(head, pos, data3);
96        // cetak node list
97        printList(head);
98    }
99 }
```

2.3.4 Hasil Output

```
<terminated> TambahSLL_2511533026 [Java Application] D:\eclipse\plugins\org.eclipse.justi.openjdk.hotspot
Senarai berantai awal: 2-->3-->5-->6
tambah 1 simpul di depan: 1-->2-->3-->5-->6
tambah 1 simpul di belakang: 1-->2-->3-->5-->6-->7
tambah 1 simpul ke data 4: Posisi di luar jangkauan1-->4-->2-->3-->5-->6-->7
```

2.3.5 Analisis

Insert di depan memiliki waktu $O(1)$, sedangkan insert di akhir dan posisi tertentu membutuhkan $O(n)$ karena harus traversal.

2.4 Class Penghapusan

2.4.1 Teori

Operasi delete meliputi penghapusan node di awal (deleteHead), di akhir (removeLastNode), dan di posisi tertentu (deleteNode).

2.4.2 Langkah-langkah

1. Implementasi deleteHead.
2. Implementasi removeLastNode.
3. Implementasi deleteNode berdasarkan posisi.

2.4.3 Contoh Kode

```
1 package hapus_2511533026;
2
3 public class HapusSL_2511533026 {
4     //fungsi untuk menghapus head
5     public static Node deleteHead(NodeSL_2511533026 head) {
6         //jika list kosong
7         if (head == null) {
8             return null;
9         }
10        //jika list tidak kosong
11        head = head.next;
12        return head;
13    }
14
15    //fungsi untuk menghapus node terakhir
16    public static Node removeLastNode(NodeSL_2511533026 head) {
17        //jika list kosong
18        if (head == null) {
19            return null;
20        }
21        //jika list tidak kosong, temp node dan return null
22        if (head.next == null) {
23            return null;
24        }
25        //jika list tidak kosong, temp node dan return null
26        NodeSL_2511533026 temp = head;
27        while (temp.next.next != null) {
28            temp = temp.next;
29        }
30        //jika list tidak kosong, temp node dan return null
31        temp.next = null;
32        return head;
33    }
34
35    //fungsi untuk menghapus node di posisi tertentu
36    public static Node deleteNode(NodeSL_2511533026 head, int position) {
37        //jika list kosong
38        if (head == null) {
39            return null;
40        }
41        //jika list tidak kosong, temp node dan return null
42        if (position == 0) {
43            return deleteHead(head);
44        }
45        //jika list tidak kosong, temp node dan return null
46        NodeSL_2511533026 temp = head;
47        for (int i = 1; i < position; i++) {
48            temp = temp.next;
49        }
50        //jika list tidak kosong, temp node dan return null
51        if (temp.next == null) {
52            return null;
53        }
54        //jika list tidak kosong, temp node dan return null
55        temp.next = temp.next.next;
56        return head;
57    }
58
59    //fungsi untuk menampilkan list
60    public static void printList(NodeSL_2511533026 head) {
61        NodeSL_2511533026 curr = head;
62        while (curr != null) {
63            System.out.print(curr.data + " ");
64            curr = curr.next;
65        }
66        System.out.println();
67    }
68
69    //fungsi untuk menampilkan list
70    public static void main(String[] args) {
71        //jika list kosong
72        NodeSL_2511533026 head = new NodeSL_2511533026(1);
73        head.next = new NodeSL_2511533026(2);
74        head.next.next = new NodeSL_2511533026(3);
75        head.next.next.next = new NodeSL_2511533026(4);
76        head.next.next.next.next = new NodeSL_2511533026(5);
77        head.next.next.next.next.next = new NodeSL_2511533026(6);
78        //jika list tidak kosong
79        printList(head);
80        //jika list tidak kosong
81        head = deleteHead(head);
82        printList(head);
83        //jika list tidak kosong
84        head = removeLastNode(head);
85        printList(head);
86        //jika list tidak kosong
87        head = deleteNode(head, position);
88        printList(head);
89        //jika list tidak kosong
90        head = deleteNode(head, position);
91        printList(head);
92    }
93 }
```

2.4.4 Hasil Output

```
<terminated> HapusSL_2511533026 [Java Application] D:\eclipse
list awal:
1-->2-->3-->4-->5-->6
List setelah head dihapus:
2-->3-->4-->5-->6
List setelah simpul terakhir di hapus:
2-->3-->4-->5
List setelah posisi 2 dihapus:
2-->4-->5
```

2.4.5 Analisis

Penghapusan node memerlukan pointer sebelumnya untuk mengubah referensi next. Perlu penanganan khusus untuk kasus node pertama dan terakhir.

2.6 Tugas Pekan 2

2.5.1 Analisis Kode Program

Analisis Class Pasien_2511533026

- Kelas ini berfungsi sebagai **node** pada linked list.
- Memiliki atribut namaPasien, keluhan, nomorAntrian, dan next.
- Menggunakan **encapsulation** dengan modifier private untuk data dan menyediakan getter serta setter (getNama, getKeluhan, getNomor, setNext).
- Konstruktor menerima nama, keluhan, dan nomor antrian sekaligus menginisialisasi pointer next menjadi null.

Analisis Class RumahSakit_2511533026

Kelas ini mengimplementasikan operasi-operasi antrian menggunakan prinsip **FIFO (First In First Out)**:

1. Daftarkan Pasien (daftarkan_3026)

- Menambahkan pasien baru di **akhir** linked list (enqueue).
- Nomor antrian otomatis bertambah menggunakan counter.
- Jika linked list kosong, pasien baru menjadi head.

2. Panggil Pasien (panggil_3026)

- Menghapus pasien di **depan** antrian (dequeue).
- Menampilkan informasi pasien yang dipanggil beserta keluhannya.

3. Tampilkan Antrian (tampilkan_3026)

- Menampilkan seluruh antrian dalam format tabel yang rapi menggunakan printf.
- Menampilkan posisi, nomor antrian, nama, dan keluhan pasien.

4. Cari Pasien (cari_3026)

- Melakukan pencarian berdasarkan nama pasien (case-insensitive).

BAB III

KESIMPULAN

Praktikum pekan ke-5 ini telah berhasil memberikan pemahaman yang komprehensif mengenai implementasi Singly Linked List secara manual. Kami telah mempelajari cara membuat node, melakukan operasi penambahan, pencarian/traversal, serta penghapusan node pada berbagai posisi. Seluruh operasi dasar SLL dapat diimplementasikan dengan baik menggunakan konsep referensi dan pointer.

Melalui praktikum ini, terlihat jelas bahwa meskipun lebih kompleks dibandingkan array, Singly Linked List menawarkan fleksibilitas yang tinggi terutama untuk operasi insert dan delete yang sering dilakukan. Pemahaman ini akan menjadi fondasi penting untuk mempelajari varian linked list lainnya seperti Doubly Linked List dan Circular Linked List pada pertemuan mendatang.

DAFTAR PUSTAKA

1. Weiss, M. A. (2014). *Data Structures and Algorithm Analysis in Java*. Pearson.
2. Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2014). *Data Structures and Algorithms in Java*. Wiley.
3. Sedgewick, R., & Wayne, K. (2011). *Algorithms*. Addison-Wesley.
4. Materi Kuliah Struktur Data – Universitas Andalas.
5. Dokumentasi Java Oracle.